

Today 11:26 AM

I am working on a Vibe Coding lab for a Python Pygame Word Scramble game.

I have a deliberate bug in the guess validation.

Current behavior:

- The game selects a secret word.
- It creates a scrambled version of that word and displays the scrambled version.
- When I enter the correct unscrambled word, the game rejects it.
- The scrambled word is incorrectly accepted as the answer.

Here is the relevant current code from game/game_engine.py:

[PASTE THE COMPLETE game_engine.py CODE HERE]

Task:

Fix ONLY the guess comparison validation bug.

Requirements:

1. The user's guess must be compared with the actual secret target word.
2. Keep the existing scoring, feedback messages, input clearing, and round progression unchanged.
3. Do not implement hints, countdown timer, or interactive letter tiles yet.
4. Do not unnecessarily rewrite the rest of the code.
5. Explain exactly what line is causing the bug and why.
6. Show the minimal code change required.

The bug is almost certainly in the validation line where the code compares the user's input against the **scrambled word** instead of the **secret target word**.

However, you haven't actually pasted the contents of `game/game_engine.py` —the placeholder `[PASTE THE COMPLETE game_engine.py CODE HERE]` is still there.

Please paste the complete file, and I'll give you:

- The exact buggy line
- Why it is wrong
- The one-line/minimal fix
- No changes to scoring, feedback, clearing, or round progression
- No hints, timer, or letter tiles
- No unnecessary refactoring

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET",
"GALAXY", "STREAM", "PUZZLE", "ALGORITHM"]
        self.secret_word = ""
        self.scrambled_word = ""

        self.score = 0
        self.feedback_msg = "Unscramble the letters above!"
        self.feedback_color = (210, 215, 225)

        self.input_box = TextBox(width // 2 - 130, 210, 160, 46)
        self.submit_btn = pygame.Rect(width // 2 + 45, 210, 95, 46)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_word = pygame.font.SysFont(None, 52)
        self.font_msg = pygame.font.SysFont(None, 26)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.next_round()

    def scramble_string(self, word):
```

```
letters = list(word)
while True:
    random.shuffle(letters)
    shuffled = "".join(letters)
    if shuffled != word or len(word) <= 1:
        return shuffled

def next_round(self):
    self.secret_word = random.choice(self.words)
    self.scrambled_word = self.scramble_string(self.secret_word)
    self.input_box.clear()

def submit_guess(self):
    guess = self.input_box.text.strip().upper()
    if not guess:
        self.feedback_msg = "Type a word before submitting!"
        self.feedback_color = (240, 170, 50)
        return

    is_correct = (guess == self.scrambled_word)

    if is_correct:
        self.score += 1
        self.feedback_msg = f"CORRECT! '{self.secret_word}' is right."
        self.feedback_color = (80, 230, 110)
        self.next_round()
    else:
        self.feedback_msg = "WRONG GUESS! Try again."
        self.feedback_color = (240, 80, 80)
        self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key ==
pygame.K_RETURN:
        self.submit_guess()
```

```
        elif event.type == pygame.MOUSEBUTTONDOWN and
event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
        pass

    def render(self, screen):
        screen.fill((26, 30, 38))

        title_surf = self.font_title.render("Word Scramble Arena", True,
(245, 245, 245))
        screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2,
25))

        score_surf = self.font_msg.render(f"Score: {self.score}", True,
(255, 220, 80))
        screen.blit(score_surf, (self.width // 2 - score_surf.get_width() //
2, 70))

        spaced_letters = " ".join(self.scrambled_word)
        scramble_surf = self.font_word.render(spaced_letters, True, (100,
200, 255))
        screen.blit(scramble_surf, (self.width // 2 -
scramble_surf.get_width() // 2, 130))

        self.input_box.render(screen)

        pygame.draw.rect(screen, (50, 150, 85), self.submit_btn,
border_radius=6)
        pygame.draw.rect(screen, (220, 220, 220), self.submit_btn,
width=2, border_radius=6)
        btn_text = self.font_btn.render("SUBMIT", True, (255, 255, 255))
        screen.blit(btn_text, (self.submit_btn.centerx -
btn_text.get_width() // 2, self.submit_btn.centery -
btn_text.get_height() // 2))
```

```

        feedback_surf = self.font_msg.render(self.feedback_msg, True,
self.feedback_color)
        screen.blit(feedback_surf, (self.width // 2 -
feedback_surf.get_width() // 2, 285))

```

The bug is exactly one line in `submit_guess()` .

Buggy line

</> Python

```
is_correct = (guess == self.scrambled_word)
```

This compares the user's guess with the **scrambled word**, not the actual secret word.

For example:

</> Plain text

```

Secret word:    PYTHON
Scrambled word: HNOPYT
User enters:    PYTHON

```

Your code checks:

</> Python

```
"PYTHON" == "HNOPYT"    # False
```

So the correct answer is rejected.

If the user enters:

</> Plain text

```
HNOPYT
```

then:

```
</> Python
```

```
"HNOPTY" == "HNOPTY"    # True
```

So the scrambled word is incorrectly accepted.

Minimal fix

Change **only this line**:

```
</> Python
```

```
is_correct = (guess == self.scrambled_word)
```

to:

```
</> Python
```

```
is_correct = (guess == self.secret_word)
```

That's it.

Your corrected `submit_guess()` is:

```
</> Python
```

```
def submit_guess(self):
    guess = self.input_box.text.strip().upper()
    if not guess:
        self.feedback_msg = "Type a word before submitting!"
        self.feedback_color = (240, 170, 50)
        return

    is_correct = (guess == self.secret_word)

    if is_correct:
        self.score += 1
        self.feedback_msg = f"CORRECT! '{self.secret_word}' is right."
        self.feedback_color = (80, 230, 110)
        self.next_round()
    else:
        self.feedback_msg = "WRONG GUESS! Try again."
        self.feedback_color = (240, 80, 80)
        self.input_box.clear()
```

What stays unchanged

- ☒ Scoring
- ☒ Correct/wrong feedback
- ☒ Input clearing
- ☒ Next-round progression
- ☒ Scrambling logic
- ☒ UI/rendering
- ☒ Keyboard/mouse submission
- ☒ No hints
- ☒ No timer
- ☒ No interactive tiles

One-line fix:

 Diff

```
- is_correct = (guess == self.scrambled_word)
+ is_correct = (guess == self.secret_word)
```

Now implement Task 2 from the README: a progressive hint system.

Current project:

- Python
- Pygame
- Word Scramble game
- The existing Task 1 guess-validation fix is already implemented.

Task 2 requirement from the README:

"Add an interactive hint feature that reveals individual letters of the secret word in their correct positions upon request, applying a proportional score penalty for each hint used."

Implement ONLY Task 2.

Requirements:

1. Add a visible HINT button to the game UI.
2. Clicking HINT should reveal one previously hidden letter of the secret word.

3. Revealed letters must appear in their correct positions.
4. Each hint should reduce the score by 0.5 points.
5. Score must never go below 0.
6. A letter that has already been revealed must not be selected again.
7. Hints should reset when a new round starts.
8. The player must still enter the complete word to submit the answer.
9. Keep the existing Task 1 validation fix unchanged.
10. Do not implement the countdown timer or interactive letter tiles yet.
11. Keep the existing game structure and modify only the files necessary.
12. Explain which files you changed and why.
13. Show the complete updated code for each changed file.

Task 2 can be implemented entirely in `game/game_engine.py`. No other file needs to change.

The key design point is that the **scrambled word remains the puzzle**, while a separate hint pattern reveals letters from `secret_word` at their original positions. The player still has to type the complete word.

Changes

- Added `HINT` button.
- Added `revealed_indices` to track which positions have been revealed.
- Each click reveals one unrevealed position.
- Deducts `0.5` points, clamped to `0`.
- Resets revealed letters on every new round.
- Hints stop working once every position is revealed.
- Task 1 validation remains `guess == self.secret_word`.
- No timer or interactive letter tiles.

Updated `game/game_engine.py`

`</>` Python

```
import random
import pygame
from game.text_box import TextBox
```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET", "GALAXY", "STREAM", "PUZZLE", "ALGORITHM"]
        self.secret_word = ""
        self.scrambled_word = ""

        self.score = 0
        self.feedback_msg = "Unscramble the letters above!"
        self.feedback_color = (210, 215, 225)

        self.input_box = TextBox(width // 2 - 130, 210, 160, 46)
        self.submit_btn = pygame.Rect(width // 2 + 45, 210, 95, 46)
        self.hint_btn = pygame.Rect(width // 2 + 150, 210, 85, 46)
```

How the hint system works

Suppose the secret word is:

</> Plain text

PYTHON

Initially:

</> Plain text

- - - - -

After one hint, for example:

</> Plain text

- Y - - -

Score:

</> Plain text

10 → 9.5

After another hint:

</> Plain text

_ Y _ H _ _

Score:

</> Plain text

9.5 → 9.0

The revealed positions are stored in:

</> Python

```
self.revealed_indices
```

So the same position cannot be selected twice.

The important part for the existing Task 1 fix is still:

</> Python

```
is_correct = (guess == self.secret_word)
```

That line has **not been changed**.

```
import random
import pygame
from game.text_box import TextBox
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
        self.width = width
        self.height = height
```

```
        self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET",
                      "GALAXY", "STREAM", "PUZZLE", "ALGORITHM"]
```

```
self.secret_word = ""
self.scrambled_word = ""

self.score = 0
self.feedback_msg = "Unscramble the letters above!"
self.feedback_color = (210, 215, 225)

self.input_box = TextBox(width // 2 - 130, 210, 160, 46)
self.submit_btn = pygame.Rect(width // 2 + 45, 210, 95, 46)

self.font_title = pygame.font.SysFont(None, 40)
self.font_word = pygame.font.SysFont(None, 52)
self.font_msg = pygame.font.SysFont(None, 26)
self.font_btn = pygame.font.SysFont(None, 24)

self.next_round()

def scramble_string(self, word):
    letters = list(word)
    while True:
        random.shuffle(letters)
        shuffled = "".join(letters)
        if shuffled != word or len(word) <= 1:
            return shuffled

def next_round(self):
    self.secret_word = random.choice(self.words)
    self.scrambled_word = self.scramble_string(self.secret_word)
    self.input_box.clear()

def submit_guess(self):
    guess = self.input_box.text.strip().upper()
    if not guess:
        self.feedback_msg = "Type a word before submitting!"
        self.feedback_color = (240, 170, 50)
    return
```

```
is_correct = (guess == self.secret_word)

if is_correct:
    self.score += 1
    self.feedback_msg = f"CORRECT! '{self.secret_word}' is right."
    self.feedback_color = (80, 230, 110)
    self.next_round()
else:
    self.feedback_msg = "WRONG GUESS! Try again."
    self.feedback_color = (240, 80, 80)
    self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key ==
pygame.K_RETURN:
        self.submit_guess()
    elif event.type == pygame.MOUSEBUTTONDOWN and
event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((26, 30, 38))

    title_surf = self.font_title.render("Word Scramble Arena", True,
(245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2,
25))

    score_surf = self.font_msg.render(f"Score: {self.score}", True,
(255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() //
```

2, 70))

```

        spaced_letters = " ".join(self.scrambled_word)
        scramble_surf = self.font_word.render(spaced_letters, True, (100,
200, 255))
        screen.blit(scramble_surf, (self.width // 2 -
scramble_surf.get_width() // 2, 130))

        self.input_box.render(screen)

        pygame.draw.rect(screen, (50, 150, 85), self.submit_btn,
border_radius=6)
        pygame.draw.rect(screen, (220, 220, 220), self.submit_btn,
width=2, border_radius=6)
        btn_text = self.font_btn.render("SUBMIT", True, (255, 255, 255))
        screen.blit(btn_text, (self.submit_btn.centerx -
btn_text.get_width() // 2, self.submit_btn.centery -
btn_text.get_height() // 2))

        feedback_surf = self.font_msg.render(self.feedback_msg, True,
self.feedback_color)
        screen.blit(feedback_surf, (self.width // 2 -
feedback_surf.get_width() // 2, 285))

```

Your current file is still the **Task 1 version**. Task 2 has not been added yet.

You only need to modify `game/game_engine.py` . Here is the complete updated file, keeping your existing structure and Task 1 fix unchanged.

game/game_engine.py

</> Python

```

import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width

```

```

self.height = height

self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET", "GALAXY", "STREAM", "PUZZLE", "ALGORITHM"]
self.secret_word = ""
self.scrambled_word = ""

self.score = 0
self.feedback_msg = "Unscramble the letters above!"
self.feedback_color = (210, 215, 225)

self.input_box = TextBox(width // 2 - 130, 210, 160, 46)
self.submit_btn = pygame.Rect(width // 2 + 45, 210, 95, 46)
self.hint_btn = pygame.Rect(width // 2 + 150, 210, 85, 46)

```

What was added

1. HINT button

</> Python

```
self.hint_btn = pygame.Rect(width // 2 + 150, 210, 85, 46)
```

2. Track revealed positions

</> Python

```
self.revealed_indices = set()
```

3. Reset hints every round

</> Python

```
self.revealed_indices = set()
```

inside next_round() .

4. Hint logic

</> Python

```
def use_hint(self):
    hidden_indices = [
        index for index in range(len(self.secret_word))
        if index not in self.revealed_indices
    ]

```

```

    ]

    if not hidden_indices:
        return

    index = random.choice(hidden_indices)
    self.revealed_indices.add(index)

    self.score = max(0, self.score - 0.5)

```

This guarantees an already-revealed position cannot be selected again.

5. Correct-position display

For PYTHON , it could show:

</> Plain text

_ Y _ _ O _

The letters come from `self.secret_word` , so they are always in their correct positions.

6. Task 1 remains unchanged

</> Python

```
is_correct = (guess == self.secret_word)
```

The player still has to type the **complete word**. The hint only reveals information; it does not automatically fill the input box.

Your previous response was incomplete and did not include the full working code.

Please provide the COMPLETE updated `game/game_engine.py` for Task 2.

Requirements:

1. Keep the existing Task 1 fix:

```
is_correct = (guess == self.secret_word)
```

2. Add a visible HINT button.
3. When the HINT button is clicked:
 - Reveal one previously unrevealed letter from `secret_word`.
 - Display it in its correct position.
 - Deduct 0.5 from the score.
 - Score must never go below 0.
 - Do not reveal the same position twice.
4. Display the hint pattern above the input, for example:
_ Y _ H _ _
5. The scrambled word must still remain visible as the puzzle.
6. The player must still type the complete answer manually.
7. Reset all revealed positions when `next_round()` starts.
8. If all letters have already been revealed, the HINT button should do nothing.
9. Add the HINT button click handling inside `handle_event()`.
10. Add the HINT button rendering inside `render()`.
11. Do NOT add the countdown timer or interactive letter tiles yet.
12. Do NOT change `text_box.py`.
13. Provide ONLY the complete `game/game_engine.py` code, properly formatted in a Python code block.

After the code, briefly list exactly what was changed.

```

import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = [
            "PYTHON",
            "PYGAME",
            "PLANET",
            "ROCKET",
            "GALAXY",
            "STREAM",
            "PUZZLE",
            "ALGORITHM"
        ]

        self.secret_word = ""

```

Changed exactly:

- Added `self.hint_btn` .
- Added `self.revealed_indices` .
- Reset revealed positions in `next_round()` .
- Added `use_hint()` with 0.5 score penalty and minimum score 0 .
- Added HINT click handling in `handle_event()` .
- Added hint-pattern rendering above the input.
- Kept the scrambled word visible.
- Kept `is_correct = (guess == self.secret_word)` unchanged.
- No changes to `text_box.py` , timer, or interactive letter tiles.

Now implement Task 3 from the README: Active Round Countdown Timer.

The current project already has:

- Task 1: fixed guess validation
- Task 2: progressive hint system
- HINT button
- revealed letter positions
- score penalty for hints

Implement ONLY Task 3.

README requirement:

"Add an active round countdown timer. Display a visible countdown timer bar that ticks down during active play. If time runs out, automatically reveal the word and advance to the next round."

Requirements:

1. Add a countdown timer for each active round.
2. Use Pygame time functionality such as `pygame.time.get_ticks()` to track elapsed time.
3. Choose a reasonable round duration, such as 15 seconds.
4. Display the remaining time visibly on the game screen.
5. Display a horizontal timer/progress bar that decreases as time runs out.
6. The timer must continuously update while the game is running.
7. When the timer reaches zero:
 - Show/reveal the current secret word in the feedback message.
 - Automatically advance to the next round.
 - Reset the timer for the new round.
 - Reset the revealed hint positions for the new round.
8. Correctly submitting an answer before the timer expires should start a fresh timer for the next round.
9. An incorrect answer should NOT advance the round and should NOT reset the timer.
10. An empty submission should NOT advance the round and should

NOT reset the timer.

11. Keep Task 1 unchanged:

`is_correct = (guess == self.secret_word)`

12. Keep Task 2 unchanged:

- HINT button
- revealed_indices
- hint pattern
- 0.5 score penalty
- score cannot go below 0

13. Do NOT implement Task 4 interactive letter tiles yet.

14. Modify only the files necessary.

15. Preserve the existing game structure and UI as much as possible.

16. Provide the COMPLETE updated code for every changed file.

17. Explain exactly what was changed and why.

Important:

Do not remove or break any existing Task 1 or Task 2 functionality.

Only `game/game_engine.py` needs to change. The existing Task 1 validation and Task 2 hint system are preserved.

`</>` Python

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = [
```

```
"PYTHON",  
"PYGAME",  
"PLANET",  
"ROCKET",  
"GALAXY",  
"STREAM",  
"PUZZLE",  
"ALGORITHM"  
]
```

Exactly what changed

1. Added a 15-second round timer

</> Python

```
self.round_duration = 15
```

2. Added round start tracking

</> Python

```
self.round_start_time = pygame.time.get_ticks()
```

3. Reset the timer in `next_round()`

</> Python

```
self.round_start_time = pygame.time.get_ticks()  
self.remaining_time = self.round_duration
```

4. Added continuous timer calculation in `update()`

- Uses `pygame.time.get_ticks()` .
- Calculates elapsed seconds.
- Updates `remaining_time` continuously.

5. Added timeout behavior

- Displays the expired word in the feedback message.
- Automatically calls `next_round()` .
- The new round resets both the timer and `revealed_indices` .

6. Added visible countdown text

</> Plain text

Time: 12.4s

7. Added horizontal progress bar

- Starts full.
- Gradually decreases toward zero.

8. Correct answer still uses Task 1

</> Python

```
is_correct = (guess == self.secret_word)
```

9. Task 2 remains intact

- HINT button
- `revealed_indices`
- Correct-position hint pattern
- 0.5 penalty
- Score cannot go below 0

10. Incorrect/empty guesses do not reset the timer.

11. No Task 4 interactive letter tiles were added.

One implementation detail to be aware of: the timeout message is immediately replaced by the new round because `next_round()` is called in the same `update()` cycle. If the README literally requires the player to see the revealed word before the next round begins, that would require a short timeout/result state rather than immediately advancing.

Hello from the pygame community.

<https://www.pygame.org/contribute.html>

PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble> git status

>> git add game/game_engine.py

>> git commit -m "Add progressive hint system"

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

(use "git push" to publish your local commits)

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)
 modified: game/game_engine.py

Untracked files:

(use "git add <file>..." to include in what will be committed)
 game/__pycache__/

no changes added to commit (use "git add" and/or "git commit -a")

[main 9762e92] Add progressive hint system

1 file changed, 182 insertions(+), 13 deletions(-)

PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
 engg\labs\Lab4\62_scramble>

Your output shows the commit **did succeed**, but there is an important issue: `game/game_engine.py` was modified again after the commit.

The key lines are:

</> Plain text

```
[main 9762e92] Add progressive hint system
1 file changed, 182 insertions(+), 13 deletions(-)
```

So the **Task 2 changes** were committed as:

</> Plain text

```
9762e92 Add progressive hint system
```

But your working tree currently has:

</> Plain text

```
modified: game/game_engine.py
untracked: game/__pycache__/
```

That means your **Task 3 timer code** is currently not committed.

Do this now

First, remove the Python cache from Git tracking/untracked files:

```
</> powershell
```

```
Remove-Item -Recurse -Force game\__pycache__
```

Then check:

```
</> powershell
```

```
git status
```

You should still see:

```
</> Plain text
```

```
modified: game/game_engine.py
```

Now commit the Task 3 changes:

```
</> powershell
```

```
git add game/game_engine.py  
git commit -m "Add active round countdown timer"
```

Then verify:

```
</> powershell
```

```
git status
```

Ideally you should get:

```
</> Plain text
```

```
On branch main  
Your branch is ahead of 'origin/main' by 2 commits.
```

```
nothing to commit, working tree clean
```

Finally, push both commits:

```
</> powershell
```

```
git push origin main
```

Your commit history should become

```
</> Plain text
```

```
9762e92 Add progressive hint system
```

```
<new-commit> Add active round countdown timer
```

One more thing: **don't commit** `__pycache__` . It contains generated Python bytecode and should normally be ignored. You should eventually add this to `.gitignore` :

```
</> gitignore
```

```
__pycache__/
```

```
*.pyc
```

If your lab requires only the Task 3 commit now, **do not use** `git add .` —stage only `game/game_engine.py` as shown above.

```
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software  
engg\labs\Lab4\62_scramble> git status
```

On branch main

Your branch is ahead of 'origin/main' by 2 commits.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

```
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
```

```
engg\labs\Lab4\62_scramble> git add game/game_engine.py
```

```
>> git commit -m "Add active round countdown timer"
```

On branch main

Your branch is ahead of 'origin/main' by 2 commits.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble>

That's correct. **Task 3 is already committed.** You tried to create another commit, but Git correctly said there was nothing new to commit.

This confirms:

</> Plain text

```
Your branch is ahead of 'origin/main' by 2 commits.  
nothing to commit, working tree clean
```

So you currently have **2 local commits** that haven't been pushed yet.

Check the commits

Run:

</> powershell

```
git log --oneline -2
```

You should see something similar to:

</> Plain text

```
<commit-id> Add active round countdown timer  
9762e92 Add progressive hint system
```

Now push them

</> powershell

```
git push origin main
```

After that, verify:

</> powershell

```
git status
```

You want:

</> Plain text

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

Don't run `git commit` again. The timer changes are already part of your second commit.

```
[main 9762e92] Add progressive hint system
1 file changed, 182 insertions(+), 13 deletions(-)
Remove-Item -Recurse -Force game\__pycache__op\5th
sem\software engg\labs\Lab4\62_scramble>
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble> git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble> git add game/game_engine.py
>> git commit -m "Add active round countdown timer"
On branch main
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)

nothing to commit, working tree clean
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble> git log --oneline -2
9762e92 (HEAD -> main) Add progressive hint system
9c713d9 Fix guess validation
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble> git push origin main
info: please complete authentication in your browser...
remote: Permission to SETAPESU26/62_scramble.git denied to
```

```
praveenjadhav585225-sketch.  
fatal: unable to access  
'https://github.com/SETAPESU26/62_scramble.git/': The requested  
URL returned error: 403  
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software  
engg\labs\Lab4\62_scramble>
```

The problem is **GitHub authentication/permission, not your Git commits.**

Your local repository is fine:

Plain text

```
9c713d9 Fix guess validation  
9762e92 Add progressive hint system
```

You are **2 commits ahead** of GitHub, but GitHub is rejecting the account you're authenticated as:

Plain text

```
praveenjadhav585225-sketch
```

while the repository belongs to:

Plain text

```
SETAPESU26/62_scramble
```

What you need to do

First check which remote you're using:

powershell

```
git remote -v
```

You should see:

Plain text

```
origin https://github.com/SETAPESU26/62_scramble.git
```

That's fine.

The real issue is that the GitHub account you're authenticated with **doesn't have write permission to that repository**.

Option 1 — If your college/team repository should allow you to push

Ask the repository owner/team admin to add your GitHub account:

```
</> Plain text
```

```
praveenjadhav585225-sketch
```

as a collaborator/member with write access.

Then authenticate again and run:

```
</> powershell
```

```
git push origin main
```

Option 2 — If you are supposed to use a different GitHub account

Your browser/Git credential is currently authenticating as the wrong account.

You need to sign into the GitHub account that has access to SETAPESU26/62_scramble , then retry:

```
</> powershell
```

```
git push origin main
```

Important: Don't reset anything

Do not run:

```
</> powershell
```

```
git reset
```

or

```
</> powershell
```

```
git reset --hard
```

Your commits are safe locally.

Your current state is:

```
</> Plain text
```

```

GitHub:
9c713d9 Fix guess validation
    ↓
9762e92 Add progressive hint system
    ↓
LOCAL main

```

The only thing preventing the push is GitHub's **403 permission error**.

Also, notice something important: your `git log` shows that **the countdown timer is NOT committed yet**. Your latest commit is still:

```
</> Plain text
```

```
9762e92 Add progressive hint system
```

So **don't assume Task 3 is in Git yet**. We should verify the actual `game_engine.py` before committing Task 3.

```
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble> git remote -v
```

```

origin https://github.com/SETAPESU26/62_scramble.git (fetch)
origin https://github.com/SETAPESU26/62_scramble.git (push)
PS C:\Users\DELL\OneDrive\Desktop\5th sem\software
engg\labs\Lab4\62_scramble>
which account shoule i use

```

Use the **GitHub account that has write access to SETAPESU26/62_scramble** .

Right now GitHub says:

 Plain text

```
Permission to SETAPESU26/62_scramble.git denied to praveenjadhav585225-sketch
```

So `praveenjadhav585225-sketch` **does not have permission to push** to that repository.

What account should you use?

If this is your college/team repository, there are two possibilities:

1. **SETAPESU26** is an **organization account** → use your personal GitHub account that was added as a collaborator/member with write access.
2. If your team specifically gave you access through another GitHub account → use that account.

The repository URL:

 Plain text

```
https://github.com/SETAPESU26/62_scramble.git
```

does **not** mean you should log in as `SETAPESU26`. You need an account that has permission to write to it.

First check

Open the repository in your browser while logged in:

Open the 62_scramble repository ↗

Check whether you can see the **Settings** tab or whether your team/organization has given your account repository access.

If you tell me **which GitHub accounts you have** (just the usernames, not passwords), I can tell you which one to use and how to switch Git authentication on Windows.

Today 5:59 PM

Implement ONLY Task 3 from the README: Active Round Countdown Timer.

The current game already has:

- Task 1: guess validation fix

- Task 2: progressive HINT system

Requirements:

1. Add a 15-second countdown for each round.
2. Display the remaining time.
3. Display a horizontal timer/progress bar that decreases as time runs out.
4. Timer must update continuously.
5. When time reaches 0:
 - Reveal the secret word in the feedback message.
 - Automatically start the next round.
 - Reset the timer.
 - Reset revealed hint positions.
6. Correct answer before timeout starts a fresh timer.
7. Wrong answer does not advance or reset timer.
8. Empty submission does not advance or reset timer.
9. Keep Task 1 unchanged:
 - `is_correct = (guess == self.secret_word)`
10. Keep Task 2 unchanged:
 - HINT button, revealed_indices, hint pattern, 0.5 penalty.
11. Do NOT implement Task 4 letter tiles.
12. Modify only necessary files.
13. Provide complete updated code for every changed file.
14. Explain the changes.

Only `game/game_engine.py` needs to change. `text_box.py` does not need modification.

</> Python

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = [
            "PYTHON",
            "PYGAME",
```

```

        "PLANET",
        "ROCKET",
        "GALAXY",
        "STREAM",
        "PUZZLE",
        "ALGORITHM"
    ]

    self.secret_word = ""

```

What changed for Task 3

- Added a **15-second round duration**.
- Added `round_start_time` using `pygame.time.get_ticks()`.
- Added continuously updating `remaining_time` in `update()`.
- Added visible `Time: X.Xs` display.
- Added a horizontal countdown/progress bar.
- Timeout automatically moves to the next round.
- `next_round()` resets both the timer and `revealed_indices`.
- Correct answers start a fresh timer.
- Wrong and empty answers leave the existing timer untouched.
- Task 1 remains exactly:

 Python

```
is_correct = (guess == self.secret_word)
```

- Task 2 HINT functionality remains intact.
- No `text_box.py` changes.
- No Task 4 letter tiles.

Implement **ONLY** Task 4 from the README: Interactive Letter Tile Sorting.

The current game already has:

- Task 1: fixed guess validation
- Task 2: progressive HINT system
- Task 3: 15-second countdown timer

Task 4 requirement:

Replace the static scrambled-letter string with interactive graphical

letter tiles that the player can click/rearrange to experiment with different sequences before submitting.

Requirements:

1. Replace the currently displayed static scrambled word with individual graphical letter tiles.
2. Each letter must appear as a separate clickable tile.
3. The player must be able to click/rearrange tiles to form a word sequence.
4. The tile arrangement should be visually clear and easy to understand.
5. The selected/rearranged tile sequence should represent the player's current answer.
6. SUBMIT should use the word formed by the tile arrangement as the guess.
7. Keep the existing text input and HINT functionality working unless the tile implementation makes the old input unnecessary. Do not break existing functionality.
8. Keep Task 1 validation unchanged:
`is_correct = (guess == self.secret_word)`
9. Keep Task 2 unchanged:
 - HINT button
 - revealed_indices
 - correct-position hint display
 - 0.5 score penalty
 - score cannot go below 0
10. Keep Task 3 unchanged:

- 15-second timer
- visible countdown
- timer progress bar
- timeout advances to next round

11. When a new round starts, reset the tile arrangement using the newly scrambled word.

12. Wrong submission should not advance the round.

13. Correct submission should award 1 point and start the next round.

14. Do not add any additional features beyond Task 4.

15. Modify only the files necessary.

16. Provide the COMPLETE code for every changed file.

17. Explain exactly how the tile interaction works and which files were changed.

Important:

Do not remove or break Tasks 1, 2, or 3.

Only `game/game_engine.py` needs to change. The simplest interaction that satisfies Task 4 without adding extra mechanics is **click two tiles to swap their positions**.

`</>` Python

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = [
```

```

        "PYTHON",
        "PYGAME",
        "PLANET",
        "ROCKET",
        "GALAXY",
        "STREAM",
        "PUZZLE",
        "ALGORITHM"
    ]

```

What changed

Only `game/game_engine.py` was changed. `text_box.py` remains untouched.

Task 4 adds:

- `self.tile_positions` — stores the current letter order.
- `self.selected_tile` — tracks the first tile selected.
- `get_tile_rect()` — calculates each tile's position.
- `handle_tile_click()` — clicking one tile and then another swaps their positions.
- `next_round()` resets the tiles to the newly scrambled word.
- `submit_guess()` now builds the guess with:

`</>` Python

```
guess = "".join(self.tile_positions).strip().upper()
```

- The static scrambled text is replaced by graphical clickable tiles.
- The selected tile is highlighted.
- The existing HINT pattern remains.
- The existing 15-second timer and progress bar remain.
- The existing HINT penalty remains.
- Task 1 remains unchanged:

`</>` Python

```
is_correct = (guess == self.secret_word)
```

The **tile order is now the actual answer submitted**. For example, if the tiles are:

`</>` Plain text

[P] [Y] [T] [H] [O] [N]

the submitted guess is PYTHON .

One caveat: the old text box is still displayed because you asked not to break existing functionality, but **Task 4 makes the tile arrangement the source of truth for submission**. Typing into the text box no longer determines the submitted guess.